Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 1

Course Outcome: CO1 **Blooms Level:L3**

Aim: Implementation of various 8 bit ALU operations (ADD, SUB, MUL, DIV, AND, OR, XOR, NOT) through assembly language programming for 8086 using MASM and Debug.

Abstract: Develop an assembly language program using the MASM (Microsoft Macro Assembler) that performs a series of arithmetic and logical operations on two predefined 8-bit unsigned numbers. The program should be structured using the SMALL memory model and should define two operands num1 and num2 as data segment variables. The operations include basic arithmetic computations such as addition, subtraction, multiplication, and division, ensuring the use of proper registers and handling of special conditions (e.g., clearing the DX register before division). Additionally, the program must demonstrate logical operations like AND, OR, XOR, and NOT to showcase bitwise manipulation of the operands. Each operation must be executed sequentially and stored in the accumulator registers, demonstrating how values are manipulated in low-level programming. The program should conclude by terminating safely using a DOS interrupt call.

Sample Input and Output:
Case 1:num1=4,num2=2
Case 2:num1=5,num2=3

Theory:
MASM (Microsoft Macro Assembler) is used to write and assemble the code. The Debug utility in DOSBox helps execute and verify results in real-time. Registers such as AX, BX, and CX are used for data manipulation.The Arithmetic Logic Unit (ALU) of the 8086 microprocessor performs arithmetic and logical operations. This experiment demonstrates the implementation of key ALU operations—ADD, SUB, MUL, DIV, AND, OR, XOR, and NOT—using 8086 assembly language. Each instruction manipulates data stored in registers or memory.

Arithmetic Operations

1. **ADD** – Add source to destination
   ADD destination, source
   Example: ADD AX, BX ; AX = AX + BX

2. **SUB** – Subtract source from destination
   SUB destination, source
   Example: SUB AX, BX ; AX = AX - BX

3. **MUL** – Unsigned multiplication
   MUL source
   Example: MUL BX ; AX = AX * BX (if AX used as default)

4. **DIV** – Unsigned division
   DIV source
   Example: DIV BL ; AL = AX / BL, AH = AX % BL

Logical Operations

5. **AND** – Logical AND between destination and source
   AND destination, source
   Example: AND AL, BL ; AL = AL AND BL

6. **OR** – Logical OR between destination and source
   OR destination, source
   Example: OR AL, BL ; AL = AL OR BL

7. **XOR** – Logical Exclusive OR
   XOR destination, source
   Example: XOR AL, BL ; AL = AL XOR BL

8. **NOT** – Logical complement (1's complement)
   NOT destination
   Example: NOT AL ; AL = NOT AL

Note:

- Destination and source can be registers or memory (but not both memory).
- For MUL and DIV, the operand is implied with AX or AL depending on operand size.

Program:

Class: SE **Subject: COA LAB** **Sem-III**

```
.MODEL SMALL
.STACK 100H
.DATA
    num1 DB 4     ; First operand
    num2 DB 2      ; Second operand

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; ----------- ADD -----------
    MOV AL, num1
    ADD AL, num2

    ; ----------- SUB -----------
    MOV AL, num1
    SUB AL, num2

    ; ----------- MUL -----------
    MOV AL, num1
    MOV AH,0
    MOV BL, num2
    MUL BL          ; AX = AX * BX
      ; Store result (for word-size MUL, result in AX)

    ; ----------- DIV -----------
    MOV AL, num1
    MOV BL, num2
    XOR DX, DX      ; Clear DX before division
    DIV BL          ; AX = AX / BX, remainder in DX


    ; ----------- AND -----------
    MOV AL, num1
    AND AL, num2


    ; ----------- OR -----------
    MOV AL, num1
    OR AL, num2


    ; ----------- XOR -----------
    MOV AL, num1
    XOR AL, num2


    ; ----------- NOT -----------
    MOV AL, num1
    NOT AL


    ; Exit program
    MOV AH, 4CH
    INT 21H

MAIN ENDP
END MAIN
```

Output:

Class: SE **Subject: COA LAB** **Sem-III**

```
Z:\>mount c c://8086
Drive C is mounted as local directory c://8086\

Z:\>c:

C:\>masm exp1.asm
Microsoft (R) Macro Assembler Version 5.00
Copyright (C) Microsoft Corp 1981-1985, 1987.  All rights reserved.

Object filename [exp1.OBJ]:
Source listing  [NUL.LST]:
Cross-reference [NUL.CRF]:

  51708 + 464836 Bytes symbol space free

      0 Warning Errors
      0 Severe  Errors
```

```
C:\>link exp1.obj

Microsoft (R) Overlay Linker  Version 3.60
Copyright (C) Microsoft Corp 1983-1987.  All rights reserved.

Run File [EXP1.EXE]:
List File [NUL.MAP]:
Libraries [.LIB]:
```

```
C:\>debug exp1.exe
-t

AX=076E  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=076F  CS=076A  IP=0003   NV UP EI PL NZ NA PO NC
076A:0003 8ED8          MOV     DS,AX
-t

AX=076E  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0005   NV UP EI PL NZ NA PO NC
076A:0005 A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0704  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0008   NV UP EI PL NZ NA PO NC
076A:0008 02060900      ADD     AL,[0009]                       DS:0009=02
```

Class: SE **Subject: COA LAB** **Sem-III**

```
AX=0706  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=000C   NV UP EI PL NZ NA PE NC
076A:000C A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0704  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=000F   NV UP EI PL NZ NA PE NC
076A:000F 2A060900      SUB     AL,[0009]                       DS:0009=02
-t
```

```
AX=0702  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0013   NV UP EI PL NZ NA PO NC
076A:0013 A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0704  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0016   NV UP EI PL NZ NA PO NC
076A:0016 B400          MOV     AH,00
-t

AX=0004  BX=0000  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0018   NV UP EI PL NZ NA PO NC
076A:0018 8A1E0900      MOV     BL,[0009]                       DS:0009=02
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=001C   NV UP EI PL NZ NA PO NC
076A:001C F6E3          MUL     BL
-t

AX=0008  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=001E   NV UP EI PL NZ NA PO NC
076A:001E A00800        MOV     AL,[0008]                       DS:0008=04
-_
```

Class: SE **Subject: COA LAB** **Sem-III**

```
AX=0008  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=001E   NV UP EI PL NZ NA PO NC
076A:001E A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0021   NV UP EI PL NZ NA PO NC
076A:0021 8A1E0900      MOV     BL,[0009]                       DS:0009=02
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0025   NV UP EI PL NZ NA PO NC
076A:0025 33D2          XOR     DX,DX
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0027   NV UP EI PL ZR NA PE NC
076A:0027 F6F3          DIV     BL
-t

AX=0002  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0029   NV UP EI PL ZR NA PE NC
076A:0029 A00800        MOV     AL,[0008]                       DS:0008=04
-
```

```
AX=0002  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0029   NV UP EI PL ZR NA PE NC
076A:0029 A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=002C   NV UP EI PL ZR NA PE NC
076A:002C 22060900      AND     AL,[0009]                       DS:0009=02
-t

AX=0000  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0030   NV UP EI PL ZR NA PE NC
076A:0030 A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0033   NV UP EI PL ZR NA PE NC
076A:0033 0A060900      OR      AL,[0009]                       DS:0009=02
-t

AX=0006  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0037   NV UP EI PL NZ NA PE NC
076A:0037 A00800        MOV     AL,[0008]                       DS:0008=04
-
```

Class: SE **Subject: COA LAB** **Sem-III**

```
AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=003A   NV UP EI PL NZ NA PE NC
076A:003A 32060900      XOR     AL,[0009]                       DS:0009=02
-t

AX=0006  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=003E   NV UP EI PL NZ NA PE NC
076A:003E A00800        MOV     AL,[0008]                       DS:0008=04
-t

AX=0004  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0041   NV UP EI PL NZ NA PE NC
076A:0041 F6D0          NOT     AL
-t

AX=00FB  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0043   NV UP EI PL NZ NA PE NC
076A:0043 B44C          MOV     AH,4C
-t

AX=4CFB  BX=0002  CX=004A  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=076E  ES=075A  SS=076F  CS=076A  IP=0045   NV UP EI PL NZ NA PE NC
076A:0045 CD21          INT     21
-
```

Conclusion:
The experiment successfully demonstrated the implementation of basic ALU operations—ADD, SUB, MUL, DIV, AND, OR, XOR, and NOT—using 8086 assembly language with MASM and Debug. The use of MASM and Debug tools provided clear insights into the low-level execution of instructions.

Exercise 1:

Aim: Implementation of various 16 bit ALU operations (ADD, SUB, MUL, DIV, AND, OR, XOR, NOT) through assembly language programming for 8086 using MASM and Debug.

Abstract: Design and implement a MASM program that performs basic 16-bit arithmetic and logical operations on two predefined hexadecimal operands:Operand 1 = 1234H and Operand 2 = 1111H. The program should execute addition, subtraction, multiplication, and division using appropriate 16-bit registers. Additionally, it must carry out logical operations including AND, OR, XOR, and NOT. Ensure proper segment initialization and use of x86 instructions to process and reflect the results in registers.